



How do we rate the Claude Code Challenge?

GridGain to Zendesk articles import challenge

How the top three teams are selected, and what to include in your submission.

Why this challenge exists

GridGain knowledge base content needs to be in Zendesk before Support can use it the way we use MariaDB content today. Your import goes into Dev first, then into Prod; so this challenge is to build real tooling, not a practice exercise.

The main goal is **learning**: building fluency with **AI-assisted coding**, and working well as a **team**. Claude Code is the engineering standard and should be the tool you use. You can use other tools to help, but the app should be built with Claude Code. The top three teams will present how they solved it at the next all-teams meeting.

Validation of the technical accuracy of the knowledge base articles is outside the scope of this challenge. Article content will be assessed to ensure the content has not changed during the import, only the formatting. Your demo should show the successful import of 1 article, but should be viable to import the full collection.

Submit a recorded demo that includes the following:

Submit demos to  Kristin Wilson before October 14.

1. Show how what you built works

Walk through the mechanism: pulling content out of Confluence, mapping it into Zendesk's article template structure, and handling formatting, attachments, and edge cases.

2. How each of you used Claude Code

Both teammates show their own AI-assisted workflow: prompts, tools, and what you accepted versus rewrote.

3. What you learned

Where AI sped you up, where it got in the way, and what you would do differently next time.

Eligibility

Your team must meet all to be considered. Use this list to check your submission before you record.

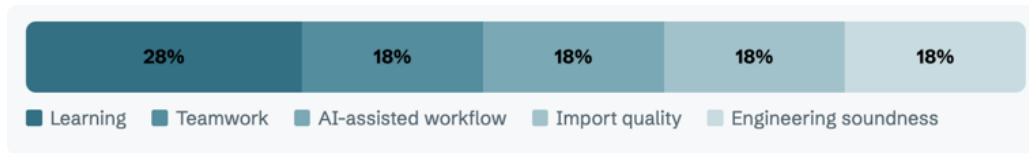
- ☐ Team of two, and each member presents their own AI-assisted workflow
- ☐ Import runs successfully in Dev, shown in a recording
- ☐ No credentials hard-coded or committed to your repository

☐ Your selected articles include at least one article with a code block

☐ Code is shared with  Kristin Wilson

Scoring

Each criterion is scored 1, 2, or 3, then weighted. Learning and reflection carries the most weight because learning is the point of the challenge.



✓ Learning 28%

How specifically your team reflects on coding with AI. I am looking for insight drawn from your own experience on this project, rather than general statements about using AI to code.

✓ Teamwork 18%

How the two of you worked together to build the solution. I am looking for how you shared the work, how you reviewed each other's contributions, and how well each of you understands the solution as a whole.

✓ AI-assisted workflow 18%

How you used AI tools to build the import. The prompts you used, why you chose your tools and models, and how you judged the quality of what AI gave you.

✓ Import quality 18%

Imported articles should reflect the original Confluence content, and not need manual rework in Zendesk.

✓ Engineering soundness 18%

How safe, reliable, and maintainable your import tool is. How you protect credentials, how the tool reports problems, and how clearly you can explain how it's built.

If teams tie

These criteria are compared in order until the tie is broken:

1. Learning and reflection

2. Teamwork
3. AI-assisted workflow
4. Import quality
5. Engineering soundness

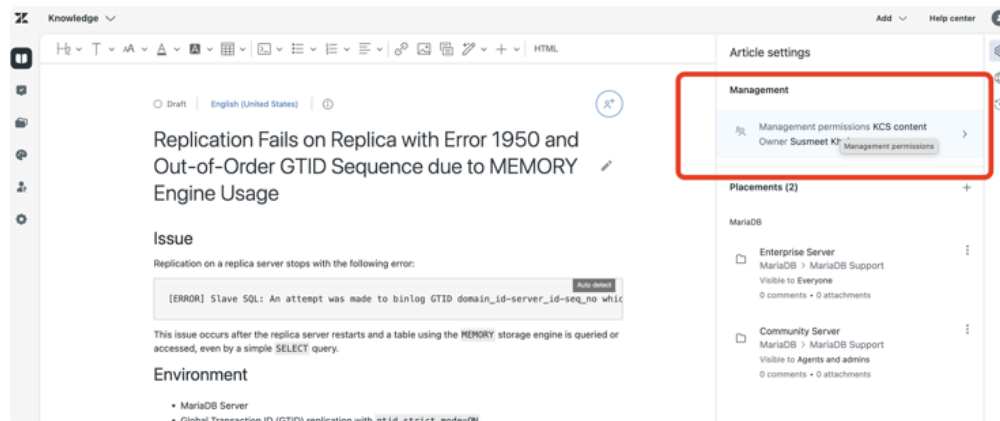
How do I get access to Zendesk Dev?

Work with  Caleb Terry or  William Fong for access.

What format do the articles need to be in?

1. Articles must align to the [Content Standards](#). This includes the sections of the article. If the original article does not contain one of the sections, then still build the section into the article and put “Not applicable” in that field.
2. Articles must be imported into Draft status.
3. Articles must be set to “[Management permissions](#)”: “KCS Content”.

Tip: Zendesk has [api details](#) available.



Informational KCS Prompt Guidance

 Sharing in case this is helpful. It is not required to use.

We use Gemini to create KCS article drafts from tickets. Below is the editorial half of the instructions the AI follows when it turns a support ticket into a KCS article.

KCS-GUIDANCE-BEGIN

```

1 AUDIENCE
2 Write for an engineer who hits the same problem but knows nothing
3 about the original reporter.
4
5 ARTICLE STRUCTURE - body_html contains exactly these four sections, in
6 this order:
7 1. Issue - the problem as the customer sees it. Quote the exact error
8 text or symptoms to make the article searchable. The main keywords

```

must appear within the first 75 words.

6 2. Environment - the explicit product and version, stated and not left to be inferred, for example "MariaDB Enterprise Server 10.6". List any configuration preconditions. Never use internal or lineage labels. A product and a version are safe to state; a hostname, database name or exact hardware specification is not.

7 3. Cause - the underlying reason. State it as a likelihood unless the ticket proves it was verified.

8 4. Resolution - specific, ordered, actionable steps as a numbered list. Every step must address the stated problem. Flag any destructive step - anything that can lose data or interrupt service - with a warning.

9

10 STYLE

11 - Plain American English with the Oxford comma. Avoid jargon, or explain it.

12 - Sentence case for the title and for every heading.

13 - Scannable: bullets and numbered steps rather than long paragraphs.

14 - Describe what the product does now. No forward-looking statements about future versions.

15 - Format literals and identifiers as <code>, and write SQL keywords in uppercase, such as NULL and INSERT. Use for placeholders the reader must substitute.

16 - Say "primary" and "replica", never "master" or "slave".

17 - Use neutral verbs such as "stop", "terminate" or "cancel", never "kill", "abort" or "hang".

18 - Never use assumptive words such as "simply", "obviously", "basically" or "easy".

KCS-GUIDANCE-END